

12. Iris Flower Classification Using KNN

Aim

To implement Iris Flower Classification using the K-Nearest Neighbours (KNN) algorithm in Python and evaluate its performance using accuracy, precision, recall, F1-score, confusion matrix, and graphs.

Software Used

Google Colab

Procedure

1. Import the required Python libraries.
2. Load the built-in Iris dataset.
3. Separate the input features and target classes.
4. Split the dataset into training and testing data.
5. Standardize the feature values.
6. Create a KNN classifier with $K = 5$.
7. Train the KNN model using the training data.
8. Predict the classes of the test data.
9. Calculate accuracy, precision, recall, and F1-score.
10. Display the confusion matrix.
11. Plot the confusion matrix and actual vs predicted classes.

Code

```
# Iris Flower Classification using KNN

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier


from sklearn.metrics import (

    accuracy_score,

    precision_score,

    recall_score,

    f1_score,

    confusion_matrix,

    ConfusionMatrixDisplay

)


# 1. Load Iris dataset

iris = load_iris()

X = iris.data

y = iris.target
```

```
# 2. Split dataset into training and testing data
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# 3. Standardize the features
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# 4. Create KNN model
```

```
knn = KNeighborsClassifier(n_neighbors=5)
```

```
# 5. Train the model
```

```
knn.fit(X_train, y_train)
```

```
# 6. Predict test data
```

```
y_pred = knn.predict(X_test)
```

```
# 7. Calculate evaluation metrics
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(

    y_test,

    y_pred,

    average="weighted"

)

recall = recall_score(

    y_test,

    y_pred,

    average="weighted"

)

f1 = f1_score(

    y_test,

    y_pred,

    average="weighted"

)

# 8. Display results

print("IRIS FLOWER CLASSIFICATION USING KNN")

print("-----")

print("K Value      :", 5)

print("Accuracy     :", round(accuracy, 4))

print("Precision     :", round(precision, 4))

print("Recall        :", round(recall, 4))

print("F1-Score      :", round(f1, 4))
```

```
# 9. Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

```
# 10. Plot Confusion Matrix
```

```
disp = ConfusionMatrixDisplay(
```

```
    confusion_matrix=cm,
```

```
    display_labels=iris.target_names
```

```
)
```

```
disp.plot()
```

```
plt.title("KNN - Iris Flower Confusion Matrix")
```

```
plt.show()
```

```
# 11. Actual vs Predicted graph
```

```
plt.figure(figsize=(8, 5))
```

```
sample_numbers = range(1, len(y_test) + 1)
```

```
plt.scatter(
```

```
    sample_numbers,
```

```
    y_test,
```

```
    label="Actual",
```

```
    marker="o"
```

```
)
```

```
plt.scatter(
```

```
    sample_numbers,
```

```
y_pred,  
  
label="Predicted",  
  
marker="x"  
  
)  
  
plt.xlabel("Test Sample")  
  
plt.ylabel("Class")  
  
plt.title("KNN - Actual vs Predicted")  
  
plt.yticks(  
  
    [0, 1, 2],  
  
    iris.target_names  
  
)  
  
plt.legend()  
  
plt.grid()  
  
plt.show()
```

OUTPUT

IRIS FLOWER CLASSIFICATION USING KNN

K Value : 5

Accuracy : 0.9333

Precision : 0.9444

Recall : 0.9333

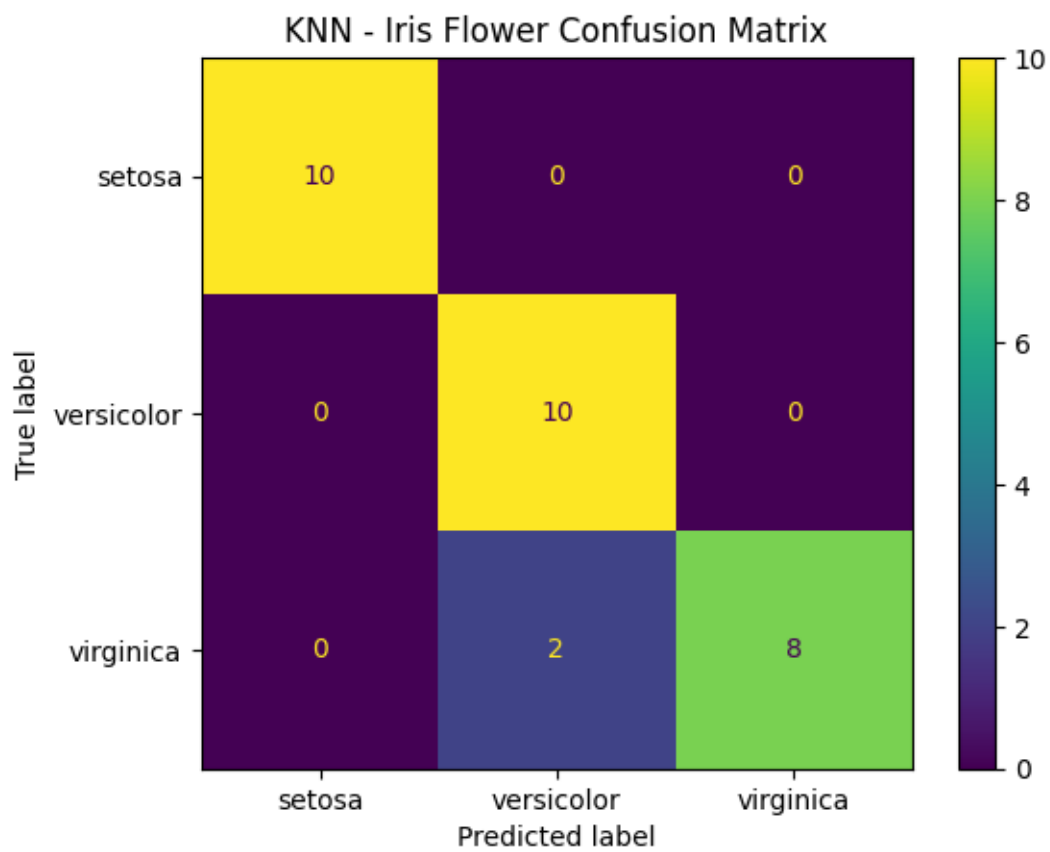
F1-Score : 0.9327

Confusion Matrix:

[[10 0 0]

[0 10 0]

[0 2 8]]



IRIS FLOWER CLASSIFICATION USING KNN

K Value : 5

Accuracy : 0.9333

Precision : 0.9444

Recall : 0.9333

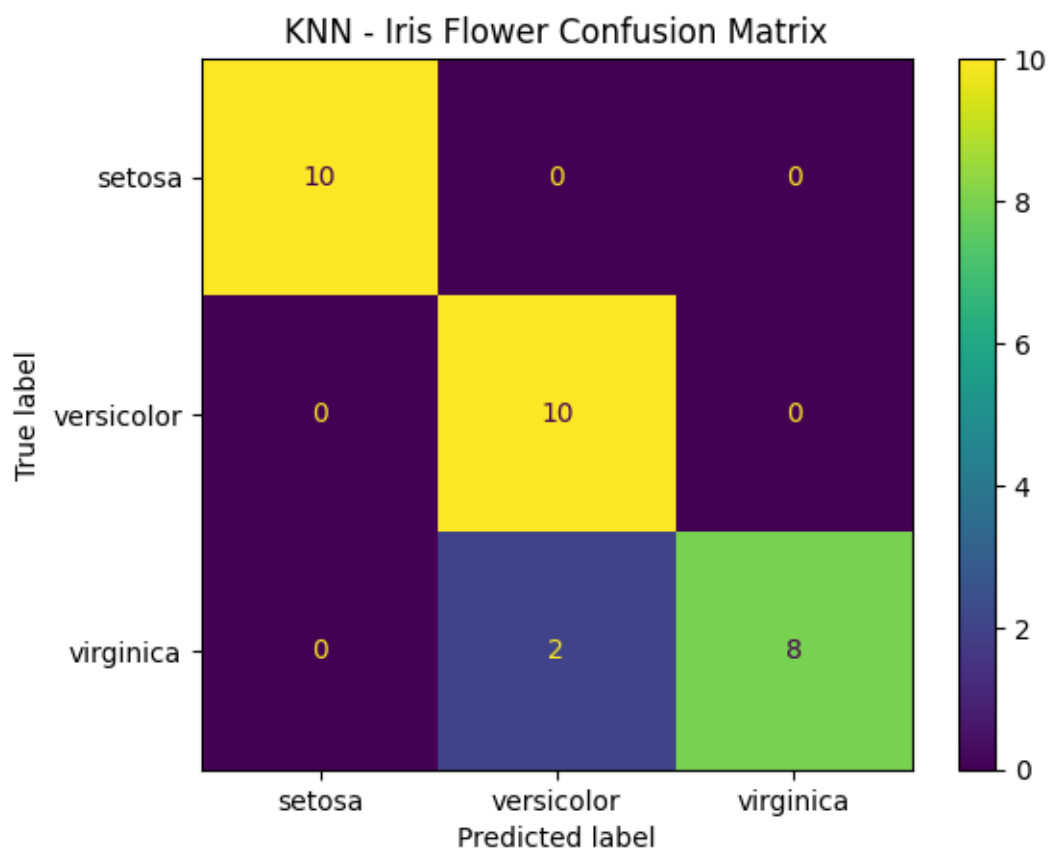
F1-Score : 0.9327

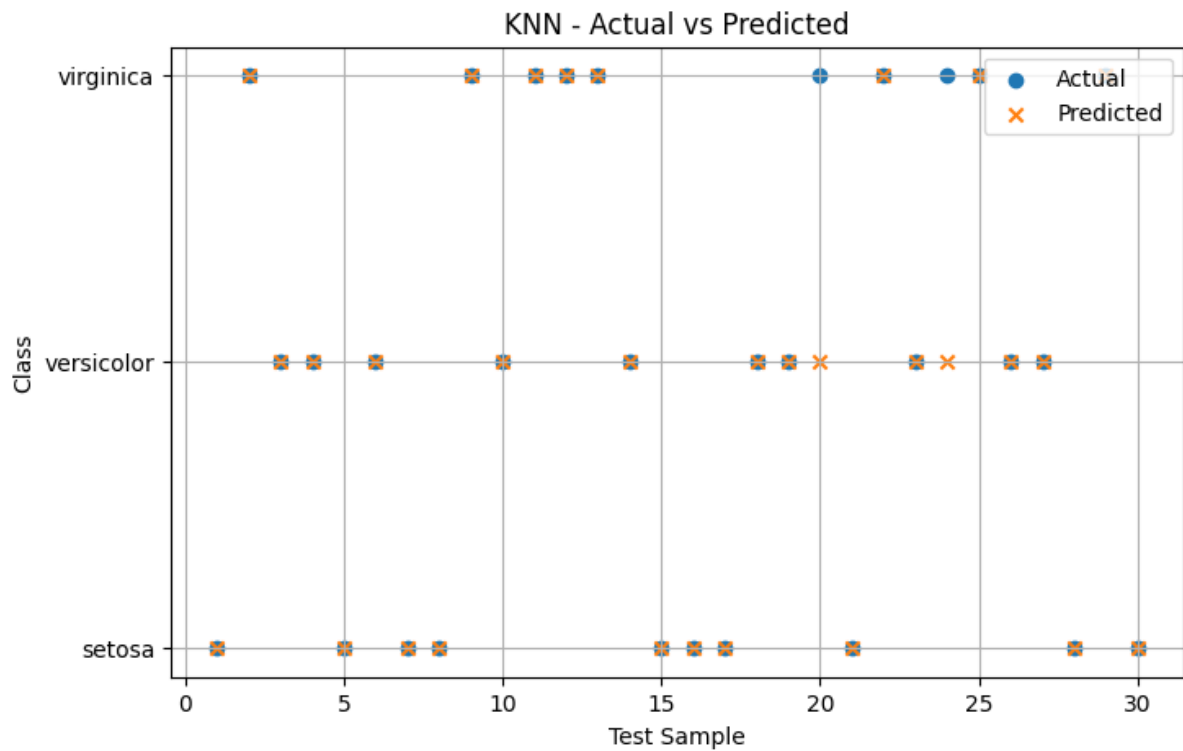
Confusion Matrix:

```
[[10 0 0]
```

```
[ 0 10 0]
```

```
[ 0 2 8]]
```





Result

Thus, Iris Flower Classification using the K-Nearest Neighbours (KNN) algorithm was successfully implemented in Python using Google Colab. The model was evaluated using accuracy, precision, recall, F1-score, and confusion matrix, and the classification results were successfully visualized using graphs.